
pynotify API Reference

Release 2.0.1

Gene C

Sep 07, 2026

PYNOTIFY API REFERENCE:

1	pynotify	1
1.1	Overview	1
1.2	Key features	1
1.3	Recent Changes	1
1.4	Getting Started	1
1.5	Mask Flags	2
2	API Reference	5
2.1	Pynotify Classes	5
3	Appendix	9
3.1	Installation	9
3.2	Dependencies	9
3.3	License	9
	Python Module Index	11

PYNOTIFY

1.1 Overview

pynotify : Python inotify implementation built atop standard C-library.

1.2 Key features

- Provides a python class to monitor 1 or more file paths for inotify events.
- All git tags are signed with arch@sapience.com key which is available via WKD or download from <https://www.sapience.com/tech>. Add the key to your package builder gpg keyring. The key is included in the Arch package and the source= line with *?signed* at the end can be used to verify the git tag. You can also manually verify the signature.

1.3 Recent Changes

2.0.1

- Manual update
- add missing py.typed to install

2.0.0

- meson / meson-python for build / package management
- periodic code review
- code cleanups and improvements.

1.4 Getting Started

A simple test program is available in the examples directory. The program monitors `/tmp/xxx` - this can be a file or a directory. To get a quick idea in one terminal do:

```
mkdir /tmp/xxx
./examples/test_inotify.py
```

In another terminal do something like:

```
touch /tmp/xxx/A
touch /tmp/xxx/B
rm /tmp/xxx/*
rmdir /tmp/xxx
```

What the code does, schematically, is:

```
from pynotify import Inotify
inotify = Inotify()
inotify.add_watch('/tmp/xxx')

for events in inotify.get_events():
    for event in events:
        if event:
            print(..)
```

The first loop uses `get_events()` method which is an iterator that returns a list of events. Each even in the list provides for:

```
event.wd          # the watch descriptor
event.mask         # the event mask
event.event_types  # list of event type enums from the mask
event.path         # the path being watched (/tmp/xxx)
event.file         # Possible subordinate file (/tmp/xxx/A)
```

Thats it in a nutshell. `add_watch` takes one optional argument:

```
Inotify.add_watch(*path*, mask=xxx)
```

`add_watch()` returns the watched descriptor, `wd`. If successful `wd` is ≥ 0 . If < 0 then it means the path is non-existent.

If not provided then the watch is for all possible event types. Where mask are event types from the enum `InotifyMask.IN_xxx`. These event elements are the same as provided in `/usr/include/sys/inotify.h`.

Before calling `get_events()` there is one additional attribute that can be set on an instance of `Inotify` which is a select timeout which defaults to 5 seconds.

```
inotify.timeout
```

The timeout is passed down to `select()` which waits on the inotify file descriptor for events to provided. If its negative then the select will wait forever, if no events occur. Otherwise the select loop will break after the timeout. A value of zero causes select to return immediately. The default value should provide reasonable behaviour.

1.5 Mask Flags

You can get the full list of possible mask flags reading code, which has comments, or using:

```
from pynotify import InotifyMask, Inotify
for item in InotifyMask.mask_to_events(0xFFFFFFFF):
    item
```

This currently outputs the following where we have manually added comments:

```

<InotifyMask.IN_ACCESS: 1>          # File was accessed
<InotifyMask.IN_MODIFY: 2>          # File was modified.
<InotifyMask.IN_ATTRIB: 4>          # Metadata changed.
<InotifyMask.IN_CLOSE_WRITE: 8>     # Writable file was closed.
<InotifyMask.IN_CLOSE_NOWRITE: 16>  # Unwritable file closed.
<InotifyMask.IN_CLOSE: 24>          # File closed
<InotifyMask.IN_OPEN: 32>           # File was opened.
<InotifyMask.IN_MOVED_FROM: 64>     # File was moved from X.
<InotifyMask.IN_MOVED_TO: 128>     # File was moved to Y.
<InotifyMask.IN_MOVE: 192>         # File was moved
<InotifyMask.IN_CREATE: 256>       # Subfile was created.
<InotifyMask.IN_DELETE: 512>       # Subfile was deleted.
<InotifyMask.IN_DELETE_SELF: 1024> # Self was deleted.
<InotifyMask.IN_MOVE_SELF: 2048>   # Self was moved.
<InotifyMask.IN_UNMOUNT: 8192>     # Backing fs was unmounted.
<InotifyMask.IN_Q_OVERFLOW: 16384> # Event queue overflowed.
<InotifyMask.IN_IGNORED: 32768>    # File was ignored.
<InotifyMask.IN_ONLYDIR: 16777216> # Only watch the path if it is a directory.
<InotifyMask.IN_DONT_FOLLOW: 33554432> # Do not follow a sym link.
<InotifyMask.IN_EXCL_UNLINK: 67108864> # Exclude events on unlinked objects.
<InotifyMask.IN_MASK_CREATE: 268435456> # Only create watches.
<InotifyMask.IN_MASK_ADD: 536870912> # Add to the mask of an already existing
↪ watch.
<InotifyMask.IN_ISDIR: 1073741824> # Event occurred against dir.
<InotifyMask.IN_ONESHOT: 2147483648> # Only send event once.
<InotifyMask.IN_ALL_EVENTS: 4095>   # All events which that can be waited on.

```


API REFERENCE

This section contains the API guide for the `pynotify` module.

2.1 Pynotify Classes

`pynotify`

```
class pynotify.Inotify
```

Bases: `InotifyBase`

Python inotify class.

Uses *inotify* from the standard C-library via `ctypes`.

```
add_watch(path: str, mask: int = <InotifyMask.IN_ALL_EVENTS: 4095>) → int
```

Adds a watch for a filesystem path.

Parameters

- `path` – The file to watch
- `mask` – The event mask to use

Returns

The watch descriptor (`wd`).

```
get_events() → Iterator[list[InotifyEvent]]
```

Wait for events. :yields: List of events.

```
rm_all_watches()
```

Remove all current watches.

```
rm_watch(path: str)
```

Remove watch on this path.

Parameters

`path` – Remove the watch for this path

```
class pynotify.InotifyBase
```

Bases: `object`

Python inotify class.

Uses *inotify* from the standard C-library via `ctypes`.

```
close()
```

Safely close the file descriptor to avoid resource leaks.

`save_errno()`

Keep the last errno

```
class pynotify.InotifyEvent(wd: int = -1, mask: int = -1, event_types: list[InotifyMask] = <factory>,
                           path: str = "", file: str = "")
```

Bases: object

One inotify event.

Holds:

- `wd` : the watch descriptor
- `mask` : the mask for this event.
- `event_types` : the list of individual event masks which are or'ed together to make the mask.
- `path` : the path to be/being watched..
- `file` : the filename.

`event_types`: list[InotifyMask]

`file`: str = ''

`mask`: int = -1

`path`: str = ''

`wd`: int = -1

```
class pynotify.InotifyMask(*values)
```

Bases: IntFlag

Mask values from `/usr/include/sys/inotify.h`. Please see the header file for description of events.

Masks can be bitwise OR together. For example `IN_CLOSE` can be written as `(IN_CLOSE_WRITE | IN_CLOSE_NOWRITE)`.

`IN_ACCESS` = 1

`IN_ALL_EVENTS` = 4095

`IN_ATTRIB` = 4

`IN_CLOSE` = 24

`IN_CLOSE_NOWRITE` = 16

`IN_CLOSE_WRITE` = 8

`IN_CREATE` = 256

`IN_DELETE` = 512

`IN_DELETE_SELF` = 1024

`IN_DONT_FOLLOW` = 33554432

`IN_EXCL_UNLINK` = 67108864

`IN_IGNORED` = 32768

IN_ISDIR = 1073741824

IN_MASK_ADD = 536870912

IN_MASK_CREATE = 268435456

IN_MODIFY = 2

IN_MOVE = 192

IN_MOVED_FROM = 64

IN_MOVED_TO = 128

IN_MOVE_SELF = 2048

IN_ONESHOT = 2147483648

IN_ONLYDIR = 16777216

IN_OPEN = 32

IN_Q_OVERFLOW = 16384

IN_UNMOUNT = 8192

classmethod mask_to_events(*mask: int*) → list[*InotifyMask*]

Return list of all events in mask

3.1 Installation

Available on

- [Github](#)
- [Archlinux AUR](#)

On Arch you can build using the provided PKGBUILD in the packaging directory or from the AUR. To build manually, clone the repo and

```
./scripts/do-build  
./scripts/do-install <destination-dir>
```

3.2 Dependencies

- Run Time :
 - python (3.14 or later)
 - glibc
- Building Package:
 - git
 - meson
 - meson-python
 - rsync

3.3 License

Created by Gene C. and licensed under the terms of the GPL-2.0-or-later license.

- SPDX-License-Identifier: GPL-2.0-or-later
- SPDX-FileCopyrightText: © 2023-present Gene C <arch@sapience.com>

PYTHON MODULE INDEX

p

pynotify, 5